

PROJECT 02

Fake News Classification & Gradio Application

End-to-End NLP Classification Project

Full Stack AI & Data Science

1 Project Overview

You will build a complete NLP classification project from exploratory analysis to a working Gradio application:

1. Explore and analyze a fake-news dataset.
2. Build leakage-safe text-classification pipelines.
3. Compare Multinomial Naive Bayes and Logistic Regression using cross-validation.
4. Evaluate the selected pipeline once on a held-out test set.
5. Build and test a Gradio interface locally.
6. Document your work on GitHub.

2 Dataset

The dataset contains the following columns:

- **Text** – Full text of the news article
- **label** – Target class: Fake or Real

Download: Use the attached `fake_and_real_news.csv` file.

Leakage-Safe Text Classification

Split the raw text into training and test sets before fitting a vectorizer. Keep `CountVectorizer` or `TfidfVectorizer` inside the scikit-learn `Pipeline` so the vocabulary and any IDF values are learned only from training data. The test set must not be used for model selection or hyperparameter tuning.

3 Project Structure

Your GitHub repository should have this structure:

```
fake-news-classifier/  
|-- data/  
|   '-- fake_and_real_news.csv  
|-- notebooks/  
|   |-- 1_eda.ipynb  
|   '-- 2_training.ipynb  
|-- app.py  
|-- models/  
|   '-- best_model.pkl  
|-- screenshots/  
|   '-- gradio_interface.png  
|-- README.md  
'-- requirements.txt
```

4 Deliverables

1. **GitHub Repository** – All code must be in a GitHub repository.
2. **1_eda.ipynb** – Exploratory Data Analysis notebook.
3. **2_training.ipynb** – Text vectorization, model comparison, and final evaluation notebook.
4. **app.py** – Gradio application file.
5. **models/** – Saved fitted classification pipeline.
6. **README.md** – Must include:
 - ▶ Project overview
 - ▶ Dataset description
 - ▶ Important EDA findings
 - ▶ Text representation and preprocessing choices
 - ▶ Model comparison table
 - ▶ Final model details and final test metrics
 - ▶ Screenshot of the Gradio web interface
 - ▶ Public Hugging Face Space URL, if deployed (optional)
 - ▶ How to run the project
7. **requirements.txt** – List of all Python packages used.

5 Part 1: Exploratory Data Analysis (1_eda.ipynb)

Your EDA notebook must include:

1. **Load and inspect data** – Display shape, data types, and the first few rows. Check for missing values and duplicate articles.
2. **Label distribution** – Count and visualize the number of Fake and Real articles. State whether the dataset is balanced or imbalanced.
3. **Text length analysis** – Create article-length features for EDA, such as word count or character count, and compare their distributions for Fake and Real articles. Summarize what you observe.
4. **Most common words** – After simple text cleaning and stopwords removal, show the top 20 most frequent words for Fake articles and Real articles separately using bar charts.
5. **Key findings** – Write a short summary of your observations:
 - ▶ Is the target balanced?
 - ▶ Are there meaningful differences in article length?
 - ▶ Which words appear frequently in each class?
 - ▶ Are there missing values, duplicates, or other data-quality issues that should be handled before training?

6 Part 2: Model Training (2_training.ipynb)

Your training notebook must include:

1. **Load data and split** – Use `Text` as the input and `label` as the target. Create a stratified 80/20 train-test split with a fixed random state.
2. **Build safe text pipelines** – Keep the vectorizer and classifier inside the same scikit-learn `Pipeline`. The vectorizer must be fitted only on the training portion of each split or cross-validation fold.
3. **Train candidate pipelines** – Compare at least the following three configurations:
 - ▶ `CountVectorizer + MultinomialNB`
 - ▶ `TfidfVectorizer + MultinomialNB`

► **TfidfVectorizer + LogisticRegression**

You may experiment with relevant vectorizer or model hyperparameters, but keep all learned text transformation inside the pipeline.

4. **Cross-validation for model selection** – Use stratified cross-validation on the training set only. Use a fixed random state when shuffling folds. For every candidate pipeline, report:
 - Mean cross-validation Accuracy
 - Standard deviation of Accuracy across folds
 - Mean cross-validation Macro F1-Score
 - Standard deviation of Macro F1-Score across folds
5. **Model comparison table** – Create one table containing all candidate pipelines and their cross-validation results. Use the table to choose the final pipeline.
6. **Select the final pipeline** – Explain why you selected it. Base the decision on cross-validation performance and stability, not on the test set.
7. **Final test evaluation** – Fit the selected pipeline on all training data, then evaluate it once on the held-out test set. Report:
 - Accuracy
 - Precision
 - Recall
 - F1-Score
 - Confusion matrix
 - Classification report

Do not use these test results to go back and choose or tune another model.

8. **Inspect predictions** – Show several correctly and incorrectly classified test articles. Briefly discuss any patterns you notice.
9. **Save the fitted pipeline** – Save the complete fitted pipeline to `models/best_model.pkl` using `joblib`. The saved object must accept raw article text and perform all required vectorization internally.

Text Preprocessing

Lowercasing is handled by scikit-learn text vectorizers by default. Stopword removal and additional cleaning are optional choices, not mandatory steps. If you use them, explain why and make sure the same preprocessing is applied consistently during training and inference.

7 Part 3: Gradio Application (app.py)

Your Gradio app must include:

1. **Load the trained pipeline** – Load `models/best_model.pkl`.
2. **Prediction function** – Accept raw news text and use the saved pipeline directly to return:
 - Predicted label: Fake or Real
 - Confidence/probability for each class
3. **Gradio interface** – Include:
 - A large text box for pasting a news article
 - Output showing the predicted label
 - Output showing confidence for both classes
 - A clear title and short description
 - At least 3 example articles or article excerpts
4. **Test locally** – Run the app with `demo.launch()` and test it with multiple examples before submission.

Inference Must Match Training

Do not create a new vectorizer inside `app.py`. Load the saved fitted pipeline and pass raw text to it. The vocabulary and model parameters used by the web app must be exactly the ones learned during training.

Optional Extension: Hugging Face Spaces

You may deploy the Gradio application to Hugging Face Spaces. If you choose to deploy it, upload `app.py`, `models/best_model.pkl`, and `requirements.txt`; test the hosted application in a fresh browser session; and include the public Space URL in your README and submission document. Hosting the application is optional and is not required to complete the project.

8 GitHub Repository Requirements

Your repository must include:

1. **README.md** – Include:
 - ▶ Project title and overview
 - ▶ Dataset description
 - ▶ Key EDA findings
 - ▶ Text representation and preprocessing choices
 - ▶ Cross-validation model comparison table
 - ▶ Final model details and test metrics
 - ▶ Confusion matrix or a short evaluation summary
 - ▶ Screenshot of the Gradio interface
 - ▶ Public Hugging Face Space URL, if deployed (optional)
 - ▶ Installation instructions
 - ▶ How to run the project
2. **Clean code** – Code should be readable, organized, and placed in the correct folders.
3. **requirements.txt** – Include all necessary packages.

9 README.md Template

Below is a template for your README.md file:

```
# Fake News Classification & Gradio Application

## Overview
[Brief description of the project]

## Dataset
- Input: Text
- Target: label (Fake / Real)
- Total samples: [number]
- Class distribution: [Fake count] / [Real count]

## Key EDA Findings
- [Finding 1]
- [Finding 2]
- [Finding 3]

## Text Representation
- [CountVectorizer / TF-IDF]
- [Any preprocessing choices you used]
```

```
## Cross-Validation Model Comparison

| Pipeline | CV Accuracy | CV Macro F1 |
|-----|-----|-----|
| CountVectorizer + MultinomialNB | [mean +/- std] | [mean +/- std] |
| TF-IDF + MultinomialNB | [mean +/- std] | [mean +/- std] |
| TF-IDF + Logistic Regression | [mean +/- std] | [mean +/- std] |

## Final Model

**Pipeline:** [selected pipeline]
**Test Accuracy:** [score]
**Test Precision:** [score]
**Test Recall:** [score]
**Test F1-Score:** [score]

**Why this model?**
[Explain why you selected this pipeline using cross-validation
performance and stability]

## Web Application

The interface is built with Gradio and can be run locally.

**Optional live app:** [your Hugging Face Space URL, if deployed]

### Screenshots

![Gradio Interface](screenshots/gradio_interface.png)

## Installation

git clone [your-repo-url]
cd fake-news-classifier
pip install -r requirements.txt

## Usage

Run the web app:
python app.py

## Project Structure

fake-news-classifier/
|-- data/
|   '-- fake_and_real_news.csv
|-- notebooks/
|   |-- 1_eda.ipynb
|   '-- 2_training.ipynb
|-- app.py
|-- models/
|   '-- best_model.pkl
|-- screenshots/
|   '-- gradio_interface.png
|-- README.md
'-- requirements.txt

## Technologies Used

- Python
- Pandas, NumPy, Matplotlib, Seaborn
- Scikit-learn
```

- Gradio
- Joblib

10 Submission

Submit to Google Classroom:

Submission Requirements

Submit one `txt`, `md`, or `pdf` document containing:

- ▶ Key findings from EDA
- ▶ Text representation and preprocessing choices
- ▶ Cross-validation model comparison table
- ▶ Final pipeline name
- ▶ Final test Accuracy and F1-Score
- ▶ GitHub repository link
- ▶ Public Hugging Face Space URL, if deployed (optional)

Example Submission Content

Example

Fake News Classification Project - [Your Name]

GitHub Repository: <https://github.com/yourusername/fake-news-classifier>

Hugging Face Space (Optional): *Add the public URL only if you deployed the application.*

Key Findings from EDA:

- ▶ *Write your first important finding.*
- ▶ *Write your second important finding.*
- ▶ *Write your third important finding.*

Text Representation / Preprocessing:

- ▶ *Describe the vectorizer used by your final pipeline.*
- ▶ *Describe any additional preprocessing choices.*

Cross-Validation Model Comparison:

Pipeline	CV Accuracy	CV Macro F1
CountVectorizer + MultinomialNB	Score	Score
TF-IDF + MultinomialNB	Score	Score
TF-IDF + Logistic Regression	Score	Score

Final Pipeline: [Selected pipeline]

Test Accuracy: [Score]

Test F1-Score: [Score]

Why this model: [Explain why cross-validation performance and stability support this choice.]

11 Setup Instructions

1. Create a GitHub repository.
2. Clone it to your local machine.
3. Create the folder structure shown above.
4. Download `fake_and_real_news.csv` to the `data/` folder.
5. Install the required packages:

```
pip install pandas numpy matplotlib seaborn scikit-learn gradio joblib nltk
```

6. Start with the EDA notebook.

Good luck!